Large Language Models

Authors: João Gabriel de Morais Bezerra e Daniel Henrique Peres Servejeira

Numerical Simulations and Artificial Intelligence Laboratory

June 10, 2026

UNIVERSIDADE ESTADUAL PAULISTA
"JÚLIO DE MESQUITA FILHO"

Language models

Remember the simple n-gram language model:

- Assigns probabilities to sequences of words;
- Generates text by sampling possible next words;
- Is trained on frequency counts computed from massive textual corpora.

Large language models share similarities, yet differ fundamentally:

- Assign probabilities to sequences of words;
- Generate text by sampling possible next words;
- Are trained by utilizing deep neural networks to learn how to guess the next word.

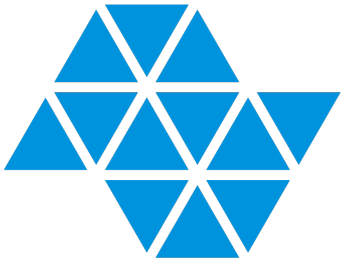
Large language models

Even though these complex models are pretrained exclusively on the simple objective of predicting the next sequential word, they implicitly learn a massive amount of useful linguistic structure, factual knowledge, and reasoning capabilities due to the astronomical scale of the text they process during training.

Three architectures for large language models

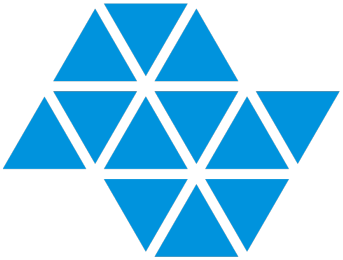
Decoders

- GPT Series
- Claude
- Llama
- Mixtral



Encoders

- BERT family
- HUBERT



Encoder-decoders

- Flan-T5
- Whisper

Encoders

Many varieties exist in the ecosystem:

- Most popular variant: Masked Language Models (MLMs) such as the BERT family.
- Trained by predicting words that have been artificially masked from the surrounding context on both the left and the right sides.
- Because they are bidirectional, they are usually finetuned (trained on supervised data) exclusively for classification and extraction tasks rather than generation.

Space for Image.
Insert the encoder diagram.

Encoder-Decoders

Trained fundamentally to map from one complete sequence to another.

Very popular for tasks requiring full context comprehension:

- Machine translation (mapping a sentence from one language entirely into another language)
- Speech recognition (mapping raw audio acoustics into written words)

Space for Image.
Insert the encoder-decoder diagram.

Large Language Models

What tasks can they do?

Big idea

A monumental breakthrough in natural language processing is the realization that many complex cognitive tasks can be mathematically reframed as simple tasks of predicting words!

This lecture: decoder-only models

Decoder-only models are commonly referred to as:

- Causal LLMs
- Autoregressive LLMs
- Left-to-right LLMs

These models function strictly by predicting words sequentially from left to right, entirely incapable of analyzing future context.

Conditional Generation

Generating text logically conditioned on previous text!

Framing lots of tasks as word prediction: Sentiment Analysis

Sentiment analysis of the string: "I like Jackie Chan"

1. We provide the autoregressive language model with this specific prefix string:
   The sentiment of the sentence "I like Jackie Chan" is:
2. We then evaluate the mathematical probability of what word it thinks comes next:

$$P(\text{positive} \mid \text{prefix}) \quad \text{versus} \quad P(\text{negative} \mid \text{prefix})$$

The model evaluates the probabilities of semantic pairs such as like/hate, love/dislike, and adore/detest to execute the classification.

Framing lots of tasks as conditional generation: QA

Question Answering (QA): "Who wrote The Origin of Species"

1. We give the language model this strict structural string:
   `Q: Who wrote the book "The Origin of Species"? A:`
2. And evaluate what word the probability distribution indicates comes next.
3. We append the result and iterate the process:
   `Q: Who wrote the book "The Origin of Species"? A: Charles`

Summarization via Conditional Generation

Original Article Example:
The only thing crazier than a guy in snowbound Massachusetts boxing up the powdery white stuff and offering it for sale online? People are actually buying it. For $89, self-styled entrepreneur Kyle Waring will ship you 6 pounds of Boston-area snow in an insulated Styrofoam box - enough for 10 to 15 snowballs, he says. But not if you live in New England or surrounding states.

Generated Summary (prompted by "tl;dr"):
Kyle Waring will ship you 6 pounds of Boston-area snow in an insulated box, enough for 10 to 15 snowballs, he says. But not if you live in New England or surrounding states. He joked about shipping the stuff to friends and family in warmer states, and an idea was born.

LLMs for summarization

Executing compression using the "Too long; didn't read" delimiter.

Sampling for LLM Generation

Decoding and Sampling

- The computational task of choosing a specific word to generate based on the model's output probabilities is called **decoding**.
- The most common method for decoding in LLMs is **sampling**.
- Sampling from a model's word distribution involves choosing random words according to the precise mathematical probability assigned by the network.
- After each token is generated, we sample subsequent words according to their probability strictly conditioned on all our previous choices.
- A Transformer language model will continually output this probability distribution over the vocabulary.

Random sampling

Algorithmic intuition for pure random sampling:

```
i = 1
w_1 \leftarrow p(w)
while w_i!= EOS:
    i = i + 1
    w_i \leftarrow p(w | w_1,..., w_{i-1})
```

- i and w_i: Track the positional index and the chosen word.
- $\leftarrow p$: Indicates drawing the word probabilistically from the distribution.
- **EOS**: The critical stop signal (End of Sequence).
- $P(w \mid \dots)$: The choice of the next word relies upon the entire historical context.

Random sampling doesn't work very well...

- Although pure random sampling generally generates sensitive, high-probability words, there are thousands of strange, low-probability words residing in the tail of the distribution.
- Each of these individual words possesses a minimal probability, but when mathematically added together, they constitute a large, dangerous part of the total probability mass.
- Therefore, under pure random sampling, they are chosen frequently enough to generate completely strange, nonsensical phrases.

Factors in word sampling: Quality vs. Diversity

Emphasizing words with medium probability introduces diversity:

- **Pros:** Results in vastly more creative and diverse textual outputs.
- **Cons:** Severely reduces quality, resulting in less factual and highly inconsistent statements.

Emphasizing words exclusively with high probability prioritizes quality:

- **Pros:** Ensures outputs are highly precise, logically coherent, and factual.
- **Cons:** Eradicates diversity, yielding boring, highly repetitive, and robotic text.

Top-k Sampling Algorithm

To truncate the dangerous probability tail, we utilize Top-k:

1. Choose a static integer set "k" representing the number of words.
2. For each word in the entire vocabulary V, use the language model to calculate the probability of that word given the contextual prefix $p(w_t \mid w_{<t})$.
3. Order the words sequentially by probability, keeping only the "k" most probable words and discarding the rest.
4. Renormalize the scores of the remaining k words so they sum to 1, creating a valid probability distribution.
5. Randomly select a word from the remaining k words, strictly according to its renormalized probability.

Top-p sampling (= nucleus sampling)

Fundamental Problem with top-k: The parameter k is fixed, meaning it may cover very different amounts of total probability mass in different contextual situations (e.g., flat vs. sharp distributions).

The Nucleus Idea: Instead of a fixed count, keep the top p percent of the probability mass.

Given a distribution $P(w_t \mid w_{<t})$, the top-p vocabulary $V(p)$ is defined mathematically as the smallest set of words such that:

$$\sum_{w \in V(p)} P(w \mid w_{<t}) \geq p$$

(Holtzman et al., 2020)

Temperature sampling (1/4)

Rather than truncating the vocabulary, we can mathematically reshape the entire distribution. This concept draws intuition from thermodynamics:

- A physical system existing at a high temperature is highly flexible, energetic, and can explore many possible variant states.
- A system at a lower temperature possesses lower energy and is mathematically likely to explore only a small subset of stable (better) states.

In low-temperature sampling configurations ($\tau \leq 1$), we smoothly force the algorithm to:

- Dramatically increase the probability of the most probable words.
- Simultaneously decrease the probability of the rare, long-tail words.

Temperature sampling (2/4)

- **t = 0.1 to 0.3 (Very Low Variance):** Exceptional for executing programming code, mathematical equations, strict translations, or answering factual queries where you desire absolute accuracy and zero creative deviation. The model will almost always greedily choose the most logical word.
- **t = 0.7 to 1.0 (Normal/High Variance):** Optimal for writing poetry, brainstorming sessions, or generating narrative stories, as the mathematical distribution allows the model to risk selecting less obvious words, generating a vastly more diverse and "creative" textual output.

Temperature sampling (3/4)

Mathematically, we divide the raw logit u by a temperature parameter τ before passing it through the final softmax activation layer.

Instead of the standard calculation:

$$y = \text{softmax}(u)$$

We compute the scaled variant:

$$y = \text{softmax}\left(\frac{u}{\tau}\right)$$

Temperature sampling (4/4)

Why does this structural adjustment work?

- When τ is exactly 1, the mathematical distribution doesn't change.
- The lower τ is, the larger the differential scores being passed to the softmax. Softmax inherently pushes high values toward 1 and low values toward 0.
- As $\tau \to 0$, the probability of the absolute most likely word asymptotically approaches 1.

Pretraining Large Language Models

Algorithms and Architectures

Pretraining Paradigm

The monumental idea driving the incredible performance metrics of modern language models:

- First, pre-train a massive Transformer model on astronomically large amounts of raw text using massive compute clusters.
- Then, apply this generalized knowledge to highly specific new tasks.

Self-supervised training algorithm

We fundamentally just train them to predict the next word!
Take a massive corpus of text. At each computational time step t...

1. Ask the neural model to mathematically predict the next word in the sequence.
2. Train the model utilizing the backpropagation of error and gradient descent to minimize the mathematical error in this prediction.

This paradigm is classified as **"Self-supervised"** learning because it does not require human annotators; it simply utilizes the natural next word in the text as the definitive ground-truth label!

Intuition of language model training: Loss

- **Loss Function Utility:** The standard metric is cross-entropy loss.
- We inherently want the neural model to assign a high probability to the true, empirical word *w*.
- Consequently, we want the calculated loss to be exceptionally high if the model assigns too low a probability to *w*.
- **CE Loss Definition:** The negative log probability that the model assigns exclusively to the true next word *w*.
- If the model assigns too low a probability to *w*, the optimizer forcefully moves the billions of model weights in the multi-dimensional direction that guarantees a higher probability assignment to *w* in the future.

Cross-entropy loss for language modeling

CE loss defines the divergence between the correct, empirical probability distribution and the model's predicted distribution:

$$L_{CE} = -\sum_{w \in V} y_t[w] \log \hat{y}_t[w]$$

The correct ground-truth distribution y_t knows the next word definitively, so it functions as a one-hot vector: it is exactly 1 for the actual next word and 0 for all other tokens.

Therefore, in this summation, all terms get multiplied by zero and disappear except one: the log probability the model assigns to the correct next word. Thus:

$$L_{CE}(\hat{y}_t, y_t) = -\log \hat{y}_t[w_{t+1}]$$

Teacher forcing technique

- At each token position t, the model processes the correct historical tokens $w_{1:t}$.
- It computes the loss (the negative log probability) for its prediction of the next token w_{t+1}.
- Critically, at the subsequent token position $t + 1$, we strictly **ignore** whatever erroneous token the model might have predicted for w_{t+1}.
- Instead, we take the correct, ground-truth word w_{t+1} from the dataset, add it securely to the context, and move on. This prevents compounding errors during training.

Training a transformer language model

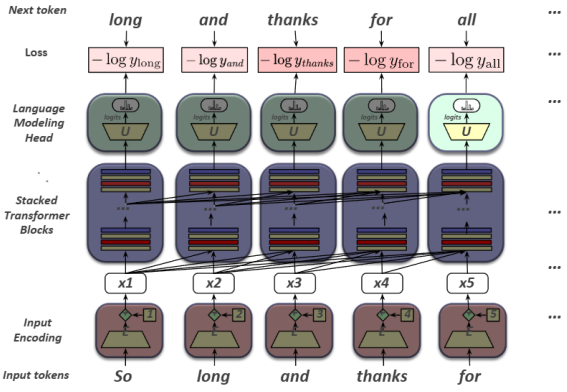


Figure: Transformer language model training scheme

$$\text{Total Batch Loss} = \frac{1}{T}\sum_{t=1}^{T} L_{CE}$$

LLMs are mainly trained on the web

- **Common Crawl:** Massive snapshots of the entire public web produced recursively by the non-profit Common Crawl organization, containing billions of raw HTML pages.
- **Colossal Clean Crawled Corpus (C4):** Authored by Raffel et al. (2020), this dataset contains 156 billion tokens of English text that have been rigorously filtered for quality.
- **What constitutes this data?** The bulk of the high-quality tokens are derived from patent text documents, Wikipedia dumps, and international news sites.

The Pile: an engineered pretraining corpus

A diverse, meticulously balanced 825 GiB open-source language modeling data set.

Academics	**Web & General**	**Books & Dialog**
PubMed Central	Pile-CC	Bibliotik
ArXiv	OpenWebText2	PG-19
FreeLaw	StackExchange	BookCorpus2
PhilPapers	Wikipedia	Ubuntu IRC / GitHub

What does a model learn from pretraining?

The raw text ingested contains an enormous amount of human knowledge. Pre-training with billions of texts containing all this knowledge is what fundamentally gives language models their ability to execute so much logic.

- Contextual Logic: "There are canines everywhere! One dog in the front room, and two dogs."
- Semantic Scaling: "It wasn't just big it was enormous."
- Factual Retrieval: "The author of 'A Room of One's Own' is Virginia Woolf."
- Mathematical Truths: "The square root of 4 is 2."

Sociotechnical problems with scraping from the web

- **Copyright Infringement:** A massive portion of the text in these datasets relies on copyrighted material. It is not clear if the fair use doctrine in the US allows for computational weight optimization. This remains a highly contentious open legal question.
- **Data Consent:** Website owners lack robust protocols to indicate they do not want their site crawled for AI training.
- **Privacy Violations:** Unfiltered websites can contain private IP addresses, social security metrics, and private phone numbers which the model may memorize.

Finetuning for Adaptation to New Domains

What happens if we fundamentally need our LLM to work well on a highly specific domain it didn't encounter sufficiently in pretraining?

- Perhaps some specific medical diagnostic phrasing or obscure legal contract domain?
- Or maybe a multilingual LM needs to see more data on a specific low-resource language that was statistically rare in the pretraining phase?

Finetuning as "continued pretraining"

- The most direct approach is to further train all the billions of parameters of the model on the new specific data.
- We deploy the exact same method (next word prediction) and mathematical loss function (cross-entropy loss) as utilized for the original pretraining.
- We treat the operation as if the new specialized data were simply appended at the tail end of the massive pretraining data sequence.
- Hence, this process is frequently classified in literature as **continued pretraining**.

Finetuning

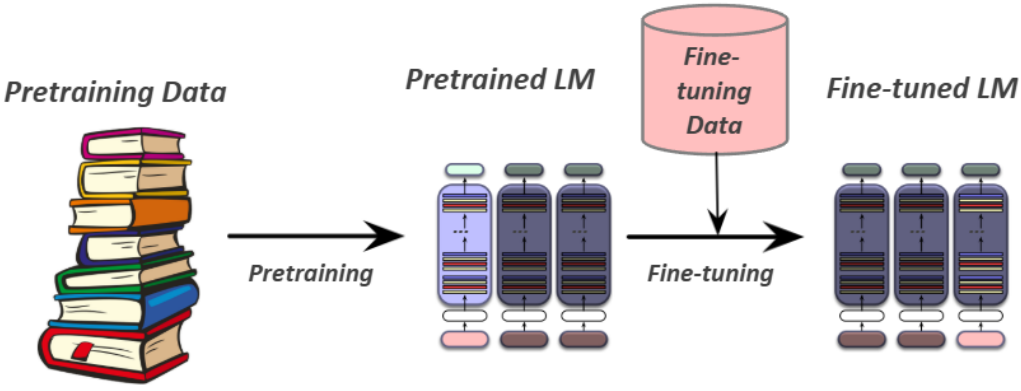


Figure: Finetuning flow

Evaluating Large Language Models

Metrics and Scaling Infrastructure

Perplexity Metric

Just as for classical n-gram grammars, the industry uses Perplexity to mathematically measure how well the LM predicts unseen text distributions.

The perplexity of a model on an unseen test set is defined as the inverse probability that the model assigns to the test set, geometrically normalized by the test set length.

For a test sequence of n tokens $w_{1:n}$, the perplexity is calculated as:

$$\text{Perplexity}(w_{1:n}) = P(w_{1:n})^{-\frac{1}{n}}$$

Why perplexity instead of raw probability?

- The raw probability metric depends entirely on the size of the test set; the probability asymptotically gets smaller the longer the text sequence runs.
- **The Superior Alternative:** A metric that operates per-word, rigorously normalized by the text length.
- Perplexity calculates the inverse probability of the test set, normalized by the total number of words. (This formulation derives directly from the cross-entropy rate in information theory).
- While probability operates on a tight range of $[0, 1]$, the perplexity range spans $[1, \infty]$.
- The higher the assigned probability of the word sequence, the lower the perplexity (indicating a vastly superior model).

Additional evaluation dimensions

- **Size Constraints:** Large models require vast arrays of GPUs and thousands of hours to train, alongside immense VRAM memory just to store the parameters.
- **Energy Usage:** Evaluators must measure the physical kilowatt-hours (kWh) consumed or kilograms of CO_2 emitted during training runs.
- **Fairness Benchmarks:** Specialized benchmarks are deployed to measure the generation of gendered and racial stereotypes, or the statistically decreased performance for language generated from or about marginalized socio-economic groups.

Scaling Laws

LLM empirical performance is strictly dependent upon:

- **Model size:** The absolute number of trainable parameters not counting the token embeddings.
- **Dataset size:** The raw volumetric amount of training data.
- **Compute:** The absolute amount of compute utilized (in FLOPs).

The performance of a large language model (measured by the loss) scales mathematically as a power-law with each of these three isolated variables:

$$L(N) = (N_c/N)^{\alpha_N}$$

$$L(D) = (D_c/D)^{\alpha_D}$$

$$L(C) = (C_c/C)^{\alpha_C}$$

Number of non-embedding parameters N

We can mathematically estimate the total parameters via:

$$N \approx 2d_{model}n_{layers}(2d_{attn} + d_{ff})$$

$$\approx 12n_{layers}d_{model}^2$$

Assuming standard dimensional ratios of $d_{attn} = d_{ff}/4 = d_{model}$.

Thus, calculating for the architecture of GPT-3, possessing $n = 96$ layers and a massive dimensionality of $d = 12288$, we determine:

$$12 \times 96 \times 12288^2 \approx 175 \text{ billion parameters.}$$

The KV Cache Bottleneck

During the pretraining phase, we can compute mathematical attention very efficiently in massive parallel batches. But this breaks down at inference! We are forced to generate the next tokens sequentially, one at a time.

- For every single new token x, we must multiply it by weight matrices W_Q, W_K, W_V to generate a query, key, and values.
- But computing the attention score requires the key and value vectors for all the prior historical tokens $x_{<i}$, yielding an $O(N^2)$ computational disaster.
- **The Engineering Solution:** Permanently store the key and value vectors in GPU VRAM memory in the KV cache, and then simply grab them from the cache for $O(N)$ inference speed.

KV Cache Matrix Architecture

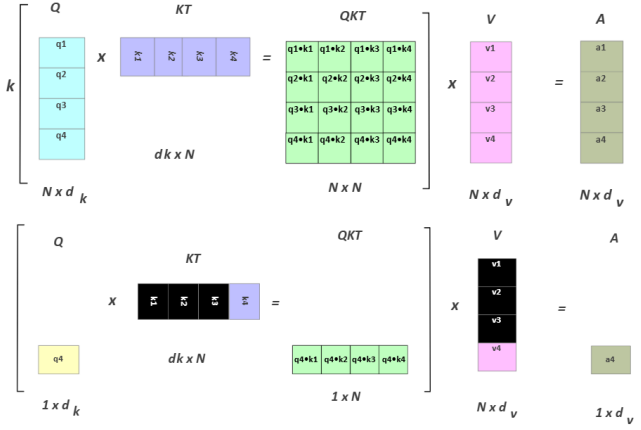


Figure: KV Chace representation

Parameter-Efficient Finetuning (PEFT)

Adapting a generalized model to a highly specific new domain by continued full pretraining presents a massive infrastructure problem with huge LLMs:

- Enormous quantities of parameters to train, requiring massive optimizer memory overhead.
- Each pass of batch gradient descent must backpropagate entirely through many massive, deep layers.
- This is extraordinarily expensive in processing power, VRAM memory, and wall-clock time.

The PEFT Paradigm:

- Efficiently select an extremely small subset of parameters to update when finetuning.
- Freeze the vast majority of the parameters (don't alter them), and only mathematically update a few specialized parameters.

LORA (Low-Rank Adaptation) Theory

- Standard Transformers consist of many dense matrix multiplication layers.
- Like the W_Q, W_K, W_V, W_O layers utilized in the attention mechanism.
- Instead of updating these massive full-rank layers during finetuning iterations...
- We permanently freeze these base layers; and we mathematically update a low-rank approximation containing drastically fewer parameters.

LORA Mathematics

Consider a base weight matrix W (shape $N \times d$) that theoretically needs to be updated during finetuning via gradient descent. Normally the updates form a dense matrix ΔW (shape $N \times d$).

In the LORA algorithm, we freeze W and update instead a low-rank decomposition of W:

- Matrix A possessing shape $N \times r$
- Matrix B possessing shape $r \times d$
- Critically, r (rank) is very small (like 1, 2, or 8)

We computationally replace $W + \Delta W$ with $W + BA$.
Forward pass: instead of $h = xW$, we calculate:

$$h = xW + xAB$$

LoRA

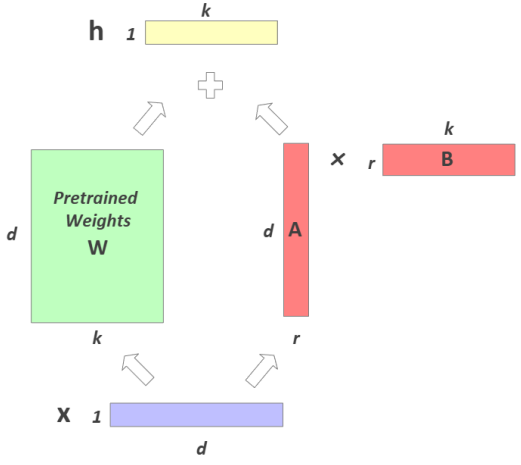


Figure: LoRA representation

Full Fine-Tuning vs. LoRA Integration

Full Fine-Tuning Profile:

- Update all model parameters ($\approx 175B$)
- Storage Requirement: Huge
- Compute Required: High
- Processing Speed: Slow

LoRA (Low-Rank Adaptation) Profile:

- Keep Pretrained Model Frozen. Add a Tiny LoRA Plugin in parallel.
- Small individual adapter generated per task ($\approx 17M$ params)
- Storage Requirement: Minimal
- Compute Required: Low
- Processing Speed: Fast

Full Fine-Tuning & LoRA

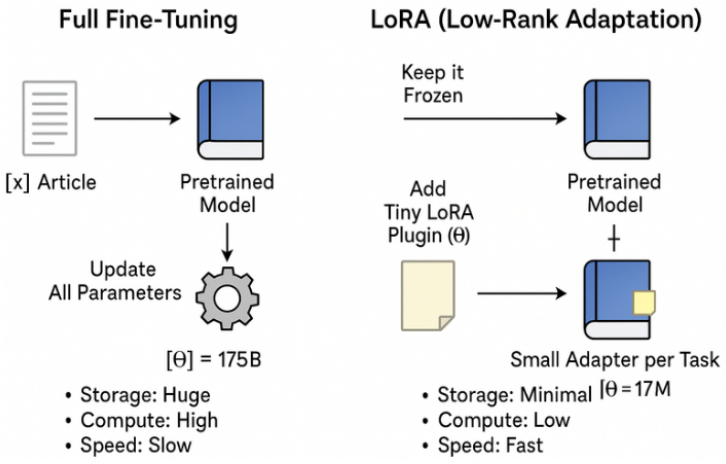


Figure: Full Fine-Tuning & LoRA flow

Societal Harms of Large Language Models

- **Hallucination**
- **Copyright Litigation**
- **Privacy Breaches**
- **Toxicity and Exploitation**
- **Political Misinformation**

Hallucination

Chatbots routinely 'hallucinate' fake corporate policies or non-existent facts. Individuals currently possess little protection or legal recourse when the technology autonomously creates and spreads falsehoods about them.

Space for Image

Insert the hallucination image from news here.

Copyright Litigation

Authors and media conglomerates are actively suing corporations, claiming mass copyright infringement regarding the hundreds of thousands of novels and news articles utilized without permission during training.

Space for Image

Insert the copyright litigation image from news here.

Privacy Breaches

The models have demonstrated the ability to leak private email addresses and phone numbers extracted from the training data.

Space for Image

Insert the privacy breaches image from news here.

Toxicity and Exploitation

Chatbots threaten users, while cleaning up the training data exacts a horrific psychological toll on human contractors forced to read descriptions of violence and abuse.

Space for Image

Insert the toxicity and exploitation image from news here.

Political Misinformation

The generation of highly persuasive false and misleading information regarding global elections.

Space for Image

Insert the political misinformation image from news here.

References I


Anthropic. 2025. Release notes: System prompts.


Bellegarda, J. R. 1997. A latent semantic analysis framework for large-span language modeling. EUROSPEECH.


Bengio, Y., R. Ducharme, and P. Vincent. 2000. A neural probabilistic language model. NeurIPS.


Bengio, Y., R. Ducharme, P. Vincent, and C. Jauvin. 2003. A neural probabilistic language model. JMLR.


Bengio, Y., et al. 2006. Neural probabilistic language models. In Innovations in Machine Learning. Springer.


Bickmore, T. W., et al. 2018. Patient and consumer safety risks when using conversational assistants... Journal of Medical Internet Research.


Blodgett, S. L., et al. 2020. Language (technology) is power: A critical survey of “bias” in NLP. ACL.


Bommasani, R., et al. 2021. On the opportunities and risks of foundation models. ArXiv.


Brown, T., et al. 2020. Language models are few-shot learners. NeurIPS.


Carlini, N., et al. 2021. Extracting training data from large language models. USENIX Security 21.


Cheng, M., E. Durmus, and D. Jurafsky. 2023. Marked personas: Using natural language prompts to measure stereotypes... ACL.

References II


Cheng, M., et al. 2025. Sycophantic ai decreases prosocial intentions and promotes dependence. ArXiv.


Coccaro, N. and D. Jurafsky. 1998. Towards better integration of semantic predictors in statistical language modeling. ICSLP.


Dai, A. M. and Q. V. Le. 2015. Semi-supervised sequence learning. NeurIPS.


Deerwester, S. C., et al. 1988. Computer information retrieval using latent semantic structure: US Patent 4,839,853.


Devlin, J., et al. 2019. BERT: Pre-training of deep bidirectional transformers for language understanding. NAACL HLT.


Dinan, E., et al. 2021. Anticipating safety issues in e2e conversational ai: Framework and tooling. ArXiv.


Dinan, E., et al. 2020. Queens are powerful too: Mitigating gender bias in dialogue generation. EMNLP.


Dodge, J., et al. 2019. Show your work: Improved reporting of experimental results. EMNLP.


Dodge, J., et al. 2021. Documenting large webtext corpora. EMNLP.


Ethayarajh, K. and D. Jurafsky. 2020. Utility is in the eye of the user: A critique of NLP leaderboards. EMNLP.

References III


Friedman, B. and D. G. Hendry. 2019. Value Sensitive Design: Shaping Technology with Moral Imagination. MIT Press.


Friedman, B., et al. 2017. A survey of value sensitive design methods. Foundations and Trends in Human-Computer Interaction.


Gao, L., et al. 2020. The Pile: An 800GB dataset of diverse text for language modeling. ArXiv.


Gehman, S., et al. 2020. RealToxicityPrompts: Evaluating neural toxic degeneration in language models. Findings of EMNLP.


Gururangan, S., et al. 2020. Don't stop pretraining: Adapt language models to domains and tasks. ACL.


Hashimoto, T., et al. 2018. Fairness without demographics in repeated loss minimization. ICML.


Heafield, K. 2011. KenLM: Faster and smaller language model queries. WMT.


Henderson, P., et al. 2020. Towards the systematic reporting of the energy and carbon footprints of machine learning. JMLR.


Henderson, P., et al. 2023. Foundation models and fair use. JMLR.


Henderson, P., et al. 2017. Ethical challenges in data-driven dialogue systems. AAAI/ACM.

References IV


Hofmann, V., et al. 2024. Ai generates covertly racist decisions about people based on their dialect. Nature.


Ischen, C., et al. 2019. Privacy concerns in chatbot interactions. International Workshop.


Jelinek, F., et al. 1975. Design of a linguistic statistical decoder for the recognition of continuous speech. IEEE.


Khattab, O., et al. 2024. DSPy: Compiling declarative language model calls into self-improving pipelines. ICLR.


Kiela, D., et al. 2021. Dynabench: Rethinking benchmarking in NLP. NAACL HLT.


Landauer, T. K., et al. 1997. How well can passage meaning be derived without using word order? COGSCI.


Liang, P., et al. 2023. Holistic evaluation of language models. Transactions on Machine Learning Research.


Liu, H., et al. 2020. Does gender matter? Towards fairness in dialogue systems. COLING.


Llama Team. 2024. The llama 3 herd of models.


Longpre, S., et al. 2024a. Consent in crisis: The rapid decline of the ai data commons. ArXiv.


Longpre, S., et al. 2024b. A pretrainer's guide to training data. NAACL HLT.

References V


McGuffie, K. and A. Newhouse. 2020. The radicalization risks of GPT-3 and advanced neural language models. ArXiv.


Mikolov, T., et al. 2013a. Efficient estimation of word representations in vector space. ICLR.


Mikolov, T., et al. 2010. Recurrent neural network based language model. INTERSPEECH.


Mikolov, T., et al. 2011. Extensions of recurrent neural network language model. ICASSP.


Mikolov, T., et al. 2013b. Distributed representations of words and phrases and their compositionality. NeurIPS.


Miller, G. A. and J. A. Selfridge. 1950. Verbal context and the recall of meaningful material.


Min, S., et al. 2022. Rethinking the role of demonstrations: What makes in-context learning work? EMNLP.


Nadeem, M., A. Bethke, and S. Reddy. 2021. StereoSet: Measuring stereotypical bias... ACL.


Neff, G. and P. Nagy. 2016. Talking to bots: Symbiotic agency and the case of Tay.


Parrish, A., et al. 2022. BBQ: A hand-built bias benchmark for question answering. Findings of ACL.


Peters, M., et al. 2018. Deep contextualized word representations. NAACL HLT.


Radford, A., et al. 2019. Language models are unsupervised multitask learners. OpenAI.

References VI


Raffel, C., et al. 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. JMLR.


Rawls, J. 2001. Justice as fairness: A restatement. Harvard University Press.


Reeves, B. and C. Nass. 1996. The Media Equation. Cambridge University Press.


Rosenfeld, R. 1992. Adaptive Statistical Language Modeling: A Maximum Entropy Approach.


Rosenfeld, R. 1996. A maximum entropy approach to adaptive statistical language modeling.


Sagawa, S., et al. 2020. Distributionally robust neural networks for group shifts. ICLR.


Shannon, C. E. 1948. A mathematical theory of communication.


Sheng, E., et al. 2019. The woman worked as a babysitter: On biases in language generation. EMNLP.


Soldaini, L., et al. 2024. Dolma: An open corpus of three trillion tokens for language model pretraining research. ArXiv.


Stolcke, A. 2002. SRILM – an extensible language modeling toolkit. ICSLP.


Strubell, E., A. Ganesh, and A. McCallum. 2019. Energy and policy considerations for deep learning in NLP. ACL.


Vaswani, A., et al. 2017. Attention is all you need. NeurIPS.

References VII


Webson, A. and E. Pavlick. 2022. Do prompt-based models really understand the meaning of their prompts? NAACL HLT.


Weizenbaum, J. 1966. ELIZA. CACM.


Wolf, M. J., et al. 2017. Why we should have seen that coming: Comments on Microsoft's Tay. The ORBIT Journal.


Xu, A., et al. 2021. Detoxifying language models risks marginalizing minority voices. NAACL HLT.


Xu, J., et al. 2020. Recipes for safety in open-domain chatbots. ArXiv.


Zao-Sanders, M. 2025. How People Are Really Using Gen AI in 2025.


Zhang, A. K., et al. 2025. Language model developers should report train-test overlap. ICML.

THANK YOU SO MUCH